



Cours : Rappels programmation Python

Pourquoi le langage Python ?

- **Facile** à apprendre avec des lignes de commande claires et concises
- C'est un langage **interprété** (et non compilé) donc plus facile à mettre en oeuvre.
- Utilisé en *open source*, pris en charge par une fondation et soutenu par une large **communauté**.
- Applications dans de divers et nombreux **domaines** (web, data science, intelligence artificielle, recherche ...)
- Utilisé dans **l'enseignement supérieur**.

Repère historique

Python a été inventé par **Guido van Rossum** Mathématicien Néerlandais.

- Il est né en 1956
- Il a inventé le langage Python en 1991
- Il a reçu le Prix du logiciel libre en 2002
- Il a travaillé pour Google et Dropbox

Monty Python's Flying Circus est une série TV humoristique britannique (1969-1974) qui lui inspiré le nom du langage.

Variables et affectation

Définition

Une **variable** sert à **stocker** une donnée, on peut penser à une boîte sur laquelle est écrit un nom et dans laquelle on place des informations diverses.
Le nom peut être n'importe quelle chaîne alphanumérique, exceptés certains mots réservés, sans commencer par un chiffre.

Python

```
# La variable a prend la valeur 2
a = 2

# La variable b prend la valeur 3
b = 3

# La variable c prend la valeur a puissance b soit donc 2 puissance 3
c = a ** b

# Ceci n'a aucun sens et renverra une erreur
2 + 3 = a

# Une instruction d'affectation
d = 2 + 3
```

Définition

Une **expression** a une **valeur** qui est le **résultat** d'une combinaison de variables ou d'objets, de constantes et d'opérateurs.

Python

```
# Ceci est une expression, la console renverra la valeur de celle-ci
2 + 3

# Ceci est une expression, la console renverra la valeur de celle-ci
```

```
0.5*d**2+1
```

```
# Ceci est une expression, La console renverra la valeur de celle-ci  
45
```

🚀 Définition

Une **instruction** est une **commande** qui doit être exécutée par la machine.

Python

```
# Ceci est une instruction, La console ne renverra rien  
# mais effectuera l'affectation de la valeur 2 dans a  
a = 2  
  
# Ceci est une instruction, La console renvoie quelque chose  
# car l'instruction demande d'afficher en console  
print(a)
```

🚀 Définition

Une **affectation** est une instruction qui **commande** à la machine de créer une variable en lui précisant son nom et sa valeur.

Python

```
# Instruction d'affectation qui met la valeur de "2+3" soit "5" dans la variable qui a pour nom a  
a = 2 + 3  
  
# Instruction d'affectation qui met la valeur de 0.5*a**2+1 dans la variable qui a pour nom x  
x = 0.5*a**2+1
```

Rappels pour affectations rapides :

- `a , b = 1,2` équivaut à `a = 1` et `b = 2`
- `x += y` équivaut à : `x = x + y`

📄 Vérification sur machine

Essayez toutes les instructions ci-dessus et observez le résultat obtenu

Types simples

🚀 Définition

Le **type** d'une variable définit l'ensemble des **valeurs** qui peuvent lui être affectées ainsi que les opérations et les fonctions utilisables.

- **int** : Les nombres entiers.
Valeurs possibles : `1` `24` `789` `-12` `-64565373567857`
- **float** : Les nombres réels. Le point remplace la virgule utilisée en mathématiques en France.
Valeurs/expressions possibles : `1.5` `2.6` `-10.5` `1.5+2.6/-10.5`
- **bool** : Les valeurs booléennes `True` (vrai) et `False` (faux).
Ces valeurs peuvent être considérées comme les entiers 1 pour `True` et 0 pour `False`.
Expressions possibles : `0 == 0` `8 + 1 == 2 * 3` `a >= 14`
- **str** : Les chaînes de caractères
Valeurs possibles : `'Bonjour'` `"la valeur de a est"` `'234'`
Pour affecter la variable de type `str` on écrira : `ch = 'bonjour'` ou `ch = "bonjour"`
Pour connaître le type d'une variable, on peut utiliser la fonction `type`

Python

```
>>># Exemple fonction type()
>>>ch = 'bonjour'
>>>print(ch)
bonjour
>>>type (ch)
<class 'str'>
```

Opérations sur les types numériques

Priorité basse	Priorité moyenne	Priorité haute	Priorité la plus haute
+ -	* / // %	**	()

$a - b + c * d ** e = a - b + (c * (d ** e))$

$5 - 8 + 4 * 2 ** 3 = 5 - 8 + (4 * (2 ** 3)) = -3 + (4 * 8) = -3 + 32 = 29$

Astuce

La méthode **PEMDAS** = **P**arenthèse **E**xposant **M**ultiplication **D**ivision **A**ddition **S**oustraction permet de retenir les priorités

Rappels :

- Le symbole `//` renvoie le quotient de la division euclidienne : $10 // 3 = 3$
- Le symbole `%` renvoie le reste de la division euclidienne (modulo) : $10 \% 3 = 1$

Python

```
>>># Quotient
>>>10 // 3
3
>>># Reste
>>>10 % 3
1
```

Comparaisons et opérateurs logiques

- `==` égal
- `!=` différent
- `>` supérieur
- `<=` inférieur ou égal
- `or` ou
- `and` et
- `not` non

Ces opérations renverront une valeur **booléenne** : `True` ou `False`

Vérification sur machine

- Affectez les valeurs `2`, `3`, `True` et `False` respectivement aux variables **a**, **b**, **c** et **d**
- Vérifiez et notez à côté la valeur des expressions suivantes :

Python

```
a == 2
a == 3
```

```
a == b

b != 3

a > 2

b <= 3

a !> 2

not c

c and d

c or d
```

Opérations sur le type str

Affectation : `ch = 'bonjour'` ou `ch = "bonjour"`

⚠ Attention

Si on écrit `ch = bonjour` alors `bonjour` est considéré comme le nom d'une variable, `ch` prendra donc la même valeur et une erreur sera signalée si cette variable n'existe pas.

- La fonction `len` renvoie le nombre de caractères contenus dans la chaîne

Python

```
>>># Fonction Len()
>>>ch = 'bonjour'
>>>len(ch)
7
>>>len("Mercredi")
8
```

- Les indices de position des caractères dans une chaîne commencent à `0`

Python

```
>>># Les indices
>>>ch = 'bonjour'
>>>ch[1]
'o'
>>>ch[0]
'b'
>>>ch[7]
Traceback (most recent call last):
  File "<basthon-input-17-9745e8801383>", line 1, in <module>
    ch[7]
    ~^^^
IndexError: string index out of range
>>>ch[1:3]
'on'
```

- Opérations sur le type `str`

Python

```
>>>a = "bon"
>>>b = "jour"
>>># Concaténation
>>>a + b
'bonjour'
>>># Multiplication
2*a
'bonbon'
```

Conversions de type

Il existe plusieurs fonctions qui permettent de forcer le type d'une variable en un autre type pour permettre de la comparer ou effectuer des opérations.

- `bool()` convertit en booléen : `0`, `""` ou `None` donnent `False` et le reste `True`
- `int()` permet de modifier une variable en entier en supprimant la partie décimale
- `float()` convertit en flottant en ajoutant une partie décimale nulle
- `str()` converti en chaîne de caractères

Python

```
# Quelques exemples de conversion de types
>>>a = 34.3
>>>b = 23
>>>type(a)
<class 'float'>
>>>type(b)
<class 'int'>
>>>c = str(a)
>>>type(c)
<class 'str'>
>>>d = float(b)
>>>type(d)
<class 'float'>
>>>e = 'Joueur 1'
>>>type(e)
<class 'str'>
>>>print("Bravo " + e + ", vous avez " + a + " points !")
Traceback (most recent call last):
  File "<basthon-input-4-7c09c1b4b3fc>", line 1, in <module>
    print("Bravo " + e + ", vous avez " + a + " points !")
    ~~~~~^~~~~~
TypeError: can only concatenate str (not "float") to str
```

Dans ce dernier exemple, l'erreur ici est de vouloir concaténer une chaîne de caractères ("Bravo Joueur 1, vous avez") avec un flottant (la variable `a`)

On corrigera avec :

Python

```
>>>print("Bravo " + e + ", vous avez " + str(a) + " points !")
Bravo Joueur 1, vous avez 34.3 points !
```

Vérification sur machine

- Vérifiez et notez à côté la valeur des expressions suivantes :

Python

```
bool(0)

bool("")

bool(None)

bool(156)

bool("toto")

bool(18.23)

int(789.956)

float(5)

str(12)
```

L'indentation

Instructions conditionnelles et boucles

Les fonctions

Espace et portée des variables

Bonnes pratiques

- Nommage des variables
- Commentaires
- Docstring